and 1 swap (in practice, as bad as a keeping an *unsorted list* of elements).

There is no easy[25] way to find a value for `D` that minimizes function `f(n) = D*logD(n)` for all values of `n`, and besides, this formulas give us just an estimate of the maximum number of accesses/swaps performed, the exact number of comparisons and swaps actually performed depends on the sequence of operations, and on the order in which elements are added.

Table 2.7 Cost of main operations provided by heaps as number of comparisons (assuming `n` elements)

Operation	Number of swaps	Extra Space
Insert	$\sim \log_D(n)$	O(1)
Top	$\sim D*\log_D(n)$	O(1)
Heapify	~n	O(1)

Then the question arises: how do we choose the best value for the branching factor?

The best we can do here is profiling our applications to choose the best value for this parameter. In theory, applications with more calls to `insert` than to `top` will perform better with larger branching factors, while when the ratio of calls between the two methods approaches 1.0, a more balanced choice will be best.